Unauthenticated Arbitrary File Write via Extension APK (Zip Slip)

Suwayomi-Server | CWE-22 Path Traversal in APK asset extraction

Vulnerability Name	Zip Slip in extension install: unauthenticated arbitrary file write leading to RCE
Researcher	Kashish Topiwala
Severity	Critical
CVSS v3.1 Base Score	9.8
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory (Zip Slip)
Affected Product	Suwayomi-Server (Suwayomi/Suwayomi-Server)
Affected Version	current (tested at commit dba836b, stable image)
Affected Component	Extension install (POST /api/v1/extension/install) -> APK asset extraction
Attacker Profile	Unauthenticated (default authMode=NONE; also drive-by via reflected-origin CORS with credentials)

Description

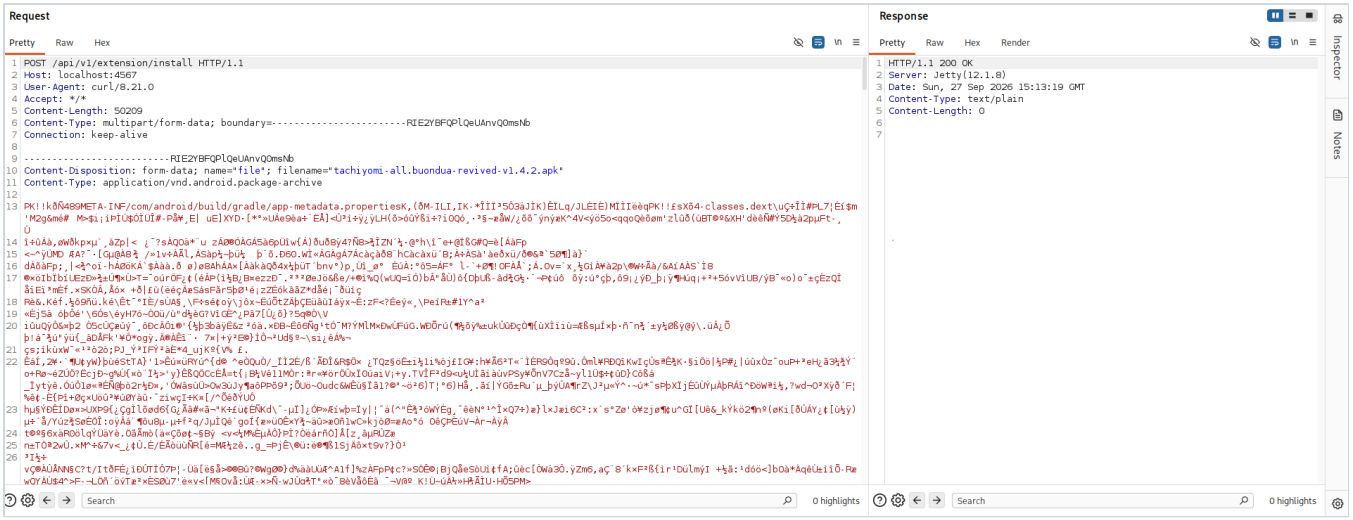
Anyone, without logging in, can upload an extension to Suwayomi. A specially crafted extension can drop a file into any location on the server, which lets the attacker place a malicious file and run their own code on the machine (full server takeover).

Affected Files

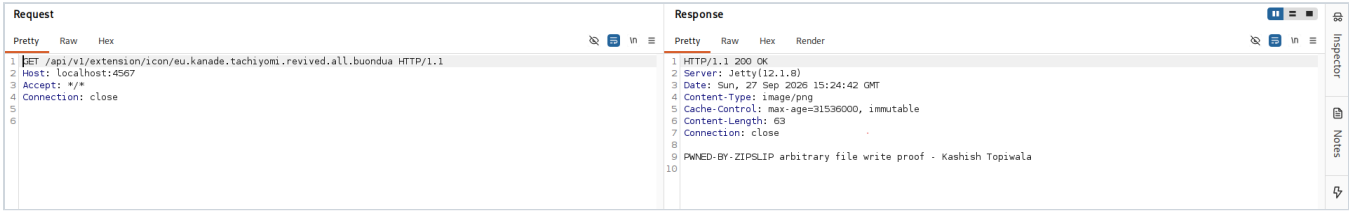
- Zip Slip sink (only guard is startsWith("assets/"); path is joined and written unnormalized): Extension.kt extractAssetsFromApk
- Install entrypoint (no auth, extracts before validating the extension): Extension.kt installExternalExtension / ExtensionPackage.Apk.prepareJarAndIcons
- Route: POST /api/v1/extension/install (ExtensionController.kt, field "file"); unauth default authMode NONE + reflect-origin CORS in JavalinSetup.kt

Steps to Reproduce

Step 1: Unauthenticated POST /api/v1/extension/install with a valid Tachiyomi extension APK (real classes.dex and manifest so it passes parsing) repackaged to include one extra entry whose name begins with assets/ followed by ../ traversal. The server returns HTTP 200 and extracts that entry during install, writing it outside the extraction directory.



Step 2: Unauthenticated GET of the planted file over HTTP (here the extension icon endpoint, which serves files from the extensions directory). The server returns the attacker-controlled bytes the crafted extension wrote (PWNED-BY-ZIPSLIP arbitrary file write proof), confirming the traversal wrote a file the attacker fully controls. The same primitive can target a startup script, a cron entry, authorized_keys, or the loaded jar to achieve remote code execution.



Impact

- An unauthenticated attacker writes a file with attacker-controlled contents to any path the server process can write, escaping the extensions directory.
- This escalates to remote code execution: write a startup/cron script, append to ~/.ssh/authorized_keys, or overwrite the extension .jar that the server loads.
- The server defaults to no authentication (authMode NONE), so no account is required; the reflect-origin, allow-credentials CORS policy also enables drive-by exploitation from a page the victim visits.
- Confidentiality, integrity, and availability are all fully impacted (host compromise).

Remediation

- Before writing each extracted asset, resolve the target against the assets root, normalize it, and verify it still starts with the assets root (reject any entry that escapes); reject entries containing .. or absolute paths.
- Do not extract or execute any part of an uploaded package before authenticating the caller; change the default authMode away from NONE and restrict CORS (do not reflect arbitrary origins with credentials).
- Validate the uploaded extension (signature/allowlist) before extraction.

References

- https://cwe.mitre.org/data/definitions/22.html
- https://security.snyk.io/research/zip-slip-vulnerability
- https://owasp.org/www-community/attacks/Path_Traversal